

MB5370

Techniques in Marine Science 1

R for Marine Science

Workbook

Version 2.0

Subject Coordinators:

Nicholas Murray

Ben Cresswell

May 2026

Introduction

0.1 About the module

Datasets and analyses in marine science are accelerating in scale, variety, and complexity to address urgent ecological changes across our tropical environments. Experience in data science is therefore among the most sought-after attributes of marine science graduates. Modern scientific investigations are also fundamentally multi-step workflows that demand

professional collaboration, rigorous data management, and absolute reproducibility to support long-term monitoring and transparent communication.

Building directly upon the foundational coding concepts established in *Programming Fundamentals* (including local Git tracking, cloud repositories, and workspace optimisation using AI tools), this module shifts your focus entirely toward advanced data science application. Students will learn to handle noisy environmental monitoring pipelines, reverse-engineer high-impact journal visualizations, manipulate native spatial datasets, and generate automated, reproducible scientific reports.

0.2 Module goals

At the completion of this module, course participants will:

- Implement advanced data engineering pipelines to clean, restructure, and validate highly volatile, real-world marine datasets.
- Master the art of publication-grade data visualization by deconstructing and replicating complex figures layer-by-layer.
- Understand coordinate reference systems (CRS) and handle vector-based spatial data frames natively within open-source programming frameworks.
- Transition from standalone script writing to compiling fully automated, reproducible dynamic reports suitable for professional scientific consulting.

0.3 Workshops schedule

This module is structured into four immersive, four-hour workshops. Each session balances direct structural scaffolding with self-directed analytical milestones.

Workshop	Focus	Core Technical Portfolio Target
Workshop 1	Foundations of Data Science — Wrangling and Plotting	Workspace projects setup, repository path calibration, manual tidyverse grammar (AI-Off), descriptive data summaries, and introductory ggplot2 layer logic using Palmer Penguins.
Workshop 2	Advanced data wrangling: Extracting ecological signals from noisy systems	Data reshaping architectures (pivoting), explicit zero engineering, transitioning from AI-Off to On, and executing un-scaffolded Data Rescue

Workshop	Focus	Core Technical Portfolio Target
		Mission.
Workshop 3	Advanced Data Communication & Open-Ended Sourcing	The Journal Plot Deconstruction Challenge and the open-ended Queensland Open Data Discovery Sprint (API data pulls and web scraping).
Workshop 4	Spatial Engineering & Reproducible Reporting	Native spatial features via the sf package, Coordinate Reference Systems (CRS), spatial joins, and migratory reporting using automated Quarto dynamic document builds.

0.4 Assessment milestone reminder

Your active development throughout this module populates your semester-long tracking portfolios. Dedicated milestones are scheduled at the end of every workshop session to secure your technical assets.

- **Assessment 1: The ePortfolio (40% of Total Grade)**
 - Platform: Google Sites.
 - Task: Requires one dedicated page per completed workshop containing your verified visual outputs, analytical scripts, and structural workflow reflections.
- **Assessment 3: The GitHub Professional Profile (20% of Total Grade)**
 - Platform: GitHub.
 - Task: Requires one clean, public-facing repository per workshop containing your full, commit-by-commit operational history.

Workshop 1: Foundations of Data Science — Wrangling and Plotting

1.1 Workshop overview and guidelines

Programming in marine biology is not merely a collection of computer code scripts; it is the modern lens through which we discover, query and interpret biological phenomena. When an ecologist records data along a coral reef transect or deploys an autonomous tracking array, they are seeking to find underlying biological patterns. However, raw data is rarely shaped in a way that allows us to immediately see these patterns!

Wrangling—the systematic pipeline of reading, inspecting, parsing, and shaping data—is how we prepare environmental attributes for formal analysis. Our ultimate objective when writing code is to look through a mass of observations and successfully [isolate the biological signal from the environmental and measurement noise](#).

To ensure you develop the mental flexibility required to manage these systems professionally, this module is built around a progressive pacing model to help build your skills both on your own and also integrating your new AI toolbox. We refer to this as the **AI-Off/On Transition**.

Today's session is conducted entirely in the **AI-Off Phase**. For the duration of this workshop, your GitHub Copilot autocomplete extensions must be completely deactivated, and your conversational chattr console interfaces must remain closed. You will write code manually, diagnose errors explicitly via the console interface, and trace memory changes step-by-step. You cannot act as a competent analytical supervisor over an automated AI coding partner until you possess deep muscle memory of how R processes data natively.

1.2 Project initialisation and workspace architecture

For this module we will establish a new, dedicated R project that is independent of our previous module's work. In your marine science careers, you will rarely start a project completely from scratch. More often, you will be joining an existing project, accessing an open-source database, or collaborating on a laboratory repository.

To safely access our clean datasets for this module's data wrangling exercises, we need to pull down the [official class repository](#). There are two primary ways to do this in GitHub: **Forking** (a connected copy) and a **Standalone Clone** (a disconnected copy).

Pathway A: The Fork (The "Connected" Copy)

Forking creates a copy of the repository on your personal GitHub account, which you can then pull to your local machine to work on yourself. However, GitHub actively remembers where it came from.

- **The Pros:** It allows you to "Pull" future updates if the original author adds more data,

and you can submit "Pull Requests" to suggest code changes back to them.

- **The Cons:** Because it remains permanently linked to the class master file, it is easy to accidentally send messy code requests to your instructor!

While Forking is a useful open-source skill you will use later in your career, **we do not recommend it for today**. You need a completely independent repository to submit for your final GitHub Portfolio Assessment.

Pathway B: The Standalone Clone (Recommended)

Instead of a connected fork, we are going to use the `usethis` package to clone the repository, remove the remote connection to the class master file, and upload a fresh, completely independent copy to your personal GitHub profile.

Step 1: Clone the class repository via RStudio

1. Navigate to the [class GitHub repository](#) in your web browser.
2. Click the green `<> Code` button and copy the HTTPS URL.
3. Open RStudio. Go to **File > New Project > Version Control > Git**.
4. Paste the Repository URL.
5. Under **Create project as subdirectory of**, click **Browse...** and visually select where you want this module saved on your hard drive.
6. Click **Create Project**.

Step 2: Remove the original connection Right now, this local project thinks its "home" (the origin remote) is the class repository on GitHub - which you do not have permission to push to. We need to delete that connection so we can set up individual projects in our own GitHub profiles. Run this in your console:

```
gert::git_remote_remove("origin")
```

Step 3: Push a fresh copy to your own profile Now you have a completely unattached local project containing all of today's `data/` and starter scripts. Let's push this up to your personal GitHub profile to serve as the foundation for your portfolio assessment. Run:

```
usethis::use_github()
```

When prompted, select the option to commit and push the files.

Exercise: Verifying your workspace organisation

Take a look at your **Files pane** (bottom right) in RStudio. You should see a basic directory structure already waiting for you, including a `data/` folder, and also a blank R script and qmd doc that you can work in for today's workshop. Click into the `data/` folder now to verify that the raw

CSV files we need for today's wrangling exercises are there. Simultaneously, check your personal GitHub profile in your web browser—you should see a brand new, independent repository hosting these exact same files. You are now safely backed up and ready to code!

Use your system file explorer or the RStudio Files pane to create additional distinct subfolders within your project root, so that you have something like the following structure:

- data/ – A strictly read-only repository for your raw, unedited source files.
- code/ – The location where your clean, sequential R tracking scripts are curated.
- outputs/ – The destination where clean data tables and high-resolution figures are saved.
- docs/ – Folder for front-facing material (e.g. Rmd/qmd docs and associated files)

Remember to make sure that you have set up:

- a .gitignore file – to manage which files you want to be left untracked (**Hint:** there is already one in the cloned repo you can use as a start point, but in future you'll need to tweak this file for your own purposes)
- A README file that will describe what this project is about and the structure/content of the repo.

Now you have a new version-controlled project for this module, **make sure you remember to save, commit and push your work to GitHub at regular intervals**, so that all your work is backed up and stored in your reproducible repo!

1.3 Memory allocation and environment hygiene

One attribute of working in R (or any object-based programming language) that you need to fully understand is where data actually exists during a live computing session. Data that exist in your environment while working in R are different from spreadsheet files and other data stored permanently on your hard drive and your computer handles file storage and live programming using two completely separate physical architectures:

- **The Disk (Permanent Storage):** This is your hard drive. Files placed here (inside your project directories) are stable and do not change unless a script explicitly over-writes them. They are slow to read but permanent.
- **The Session Environment (RAM / Computer Memory):** When you execute an import command in R, your computer reads the raw text entries from your disk, parses them line-by-line, and builds a fluid, temporary digital copy inside your Random Access Memory (RAM). All your subsequent filters, mathematical mutations, and statistical summaries take place inside this volatile memory block. If your computer loses power or you close RStudio without explicitly writing your updated objects back to the disk, every modification

you made vanishes instantly.

Because your computer's active memory can retain hidden data values across multiple computing runs, old variables can linger in your background environment and cause silent, un-trackable errors in your current analysis. To eliminate this issue, we can establish a baseline workflow at the top of every fresh script file, using:

```
# Housekeeping
## Inventory every active object currently residing in session RAM
objects()
## Purge global environment
rm(list = ls())
# Confirm that global session memory now completely vacant
objects()
```

Best practice: breaking the "Save Workspace" reflex and purging .RData

When you attempt to close RStudio or exit a project, your computer may present you with a pop-up dialog box asking: "*Save workspace image to .RData?*" Our universal psychological instinct when a computer asks us if we want to save our work is to immediately hit **Yes**. In data science, you must train yourself to break this habit and explicitly choose **No**.

If you select "Yes", RStudio saves a hidden binary snapshot of your current temporary RAM directly into your project folder, in the form of an .RData file. The next time you open this project, RStudio quietly reloads all those old, stale variables, data frames, and broken code attempts straight back into your memory before you even run a single line of your script. This creates a massive trap for reproducible science: your code might run perfectly on your machine because it is relying on a hidden, old piece of "junk" data floating in the background, but it will instantly break when your co-authors, advisors, or journal reviewers try to run your script on their computers.

Remember this golden rule: **As long as you have saved your raw code scripts, you have saved everything**. If your scripts are written correctly from an empty environment, you can re-create your entire analytical pipeline, all intermediate objects, and every high-resolution figure from scratch using that code.

To permanently adjust your settings so RStudio protects you from this trap, execute the following steps right now:

1. In the top menu bar of RStudio, navigate to **Tools** → **Global Options...**
2. In the **General** tab (under the *Basic* section), locate the **Workspace** heading.
3. Uncheck the box that says: ☐ Re-restore .RData into workspace at startup

4. Find the dropdown menu next to **Save workspace to .RData on exit:** and change it from *Ask* or *Always* to **Never**.
5. Click **Apply** and then **OK**.

Finally, if you have already accidentally accumulated background junk files from previous sessions, you need to evict them from your physical hard drive. Look at the **Files** pane in the bottom-right corner of your RStudio screen. If you see a file explicitly named `.RData`, check the box next to it, click the delete icon, and purge it for good.

Advanced Troubleshooting Note: If you ever open RStudio cleanly (without opening a specific Project) and notice that variables or data frames are already mysteriously floating in your Environment tab, you have accidentally saved an `.RData` file to your computer's global user home directory.

You can wipe this out entirely from inside R by executing this line in your console:

```
unlink("~/RData")
```

This tells R to look at your root user home directory (~) and permanently remove (unlink) the rogue configuration file, ensuring a clean slate even when you aren't working inside a dedicated project directory (but you shouldn't be doing this anymore!)

1.4 Ingesting data into R

For most data analysis in marine science, we need to load or import datasets from our hard drives into R's session memory. While historical base R documentation relies on legacy commands like `read.csv()`, modern environmental analytics utilises the optimised, speed-stable tools built into the `readr` and `readxl` libraries.

Permanent data files are fundamentally long, flat strings of plain text. To project these strings into a two-dimensional tabular grid, individual data fields must be separated by an explicit character called a **delimiter**.

- **CSV (Comma-Separated Values):** Cell values are partitioned by commas (,). This is the universal standard for scientific data exchange.
- **TSV (Tab-Separated Values):** Cell values are partitioned by a standard horizontal tab whitespace (\t). This format is frequently encountered when downloading data streams from automated oceanographic instrumentation or telemetry loggers.
- **Excel Spreadsheets (.xlsx / .xls):** Complex binary structures that contain multiple hidden layout sheets, proprietary metadata formatting, and human-readable style definitions.

They cannot be parsed as plain text strings.

Open the blank R script inside your code/ folder, and practice reading these varied inputs. Note you will need to install the [here\(\)](#) package!

```
# Load the primary data science framework and Excel import library
library(tidyverse)
library(readxl)

# Practice Import A: Loading a standard comma-separated plain text file
benthic_cover <- read_csv(here::here("data/reef_cover_log.csv"))

# Practice Import B: Parsing a tab-separated telemetry instrument array string
acoustic_stream <- read_tsv(here::here("data/acoustic_telemetry_stream.txt"))

# Practice Import C: Targeting a specific sheet in a multi-tab Excel spreadsheet
fisheries_annual <- read_excel(here::here("data/fish_catch_data.xlsx"), sheet =
"Commercial_2026")
```

In the field or the lab, as biologists we may record notes or conversational summary lines directly into the top of Excel data sheets before the actual column rows begin (Note: you should try to avoid this wherever possible!!). If you try to read these files naively, R will treat those conversational titles as your database column headers, instantly corrupting your entire data ingestion pipeline.

Try loading this dataset:

```
# Read in mangrove_data
mangrove_data <- read_csv(file = here::here("data/mangrove_survey_raw.csv"))
```

What happened when you tried to do this? You can open the actual csv file in Excel or similar program to see what we are trying to import. Sometimes we can't avoid dealing with messy datasets with these sorts of challenges, so we need an approach that can handle all the field notes at the top, as well as the various different ways of recording "NA". Luckily, we can modify import instructions using structural arguments:

```
# Use args within read_csv to skip headers and declare missing flags
mangrove_data <- read_csv(
  here::here("data/mangrove_survey_raw.csv"),
  skip = 5,    # Skip the first 5 lines of field notes
  na = c(".", "NA", "9999", "ND", "blank")) # Convert known text alts to true NA
```

By providing the na character vector, you instruct R to search for common biological placeholder shorthands (like . or "ND" for No Data) and normalise them into an explicit, trackable missing indicator state (NA) right at the point of ingestion. This can be done later in the data wrangling process, but good to do it now if you know it needs to be done.

1.5 Data frame architectures: Tibbles versus legacy tables

When you import tabular assets using modern tidyverse commands, the resulting object is stored in your environment memory as a specialised structure called a **tibble** (tbl_df). While a tibble is still fundamentally a base R **data frame** (data.frame), its underlying software architecture has been optimized to prevent common data corruption errors and console performance issues.

Understanding how these two data structures behave differently under pressure is essential for error diagnostics:

Functional Behaviour	Legacy Base R data.frame	Modern Tidyverse tibble
Console Output	Will attempt to print every single row and column to your terminal screen, easily causing your interface to freeze if the table is large.	Limits console printing to the first 10 rows and fits columns to the width of your screen, summarizing hidden rows cleanly.
Vector Verification	Prints only the text names of column fields. You must run independent functions to view variable attributes e.g. str() or class()	Explicitly prints the variable vector type directly beneath every column name (e.g., <chr>, <dbl>, <fct>).
Partial Matching Trap	If a column named salinity_ppt is present, calling df\$sal will silently extract the column without warning.	Disallows partial matching completely. Calling tb\$sal returns a clear NULL warning flag, exposing structural gaps immediately.

To see this behavioral difference directly on your machine, initialise a native tidyverse data object and enforce a comparative class transformation:

```
# Force a modern tibble to degrade into a legacy base R data frame structure
```

```
benthic_cover_df <- as.data.frame(benthic_cover)
```

And compare...

```
# Print the old-style dataframe structure to view
print(benthic_cover_df)
# And compare with tibble alternative
print(benthic_cover)
```

Observe your console feedback. The tibble structure immediately tells you its dimensions and highlights variable configurations right below the column headers. This constant visual validation is your primary defense against typing errors.

1.6 Wrangling out ecological signals using Palmer Penguins

To explore how data wrangling reveals information about underlying ecology, we will transition to working with a landmark built-in dataset: the Palmer Penguins archive. This collection contains rigorous multi-species morphological tracking records collected at the Palmer Station long-term ecological research site in Antarctica. You can get more about Palmer Station [here](#) and even [read some actual penguin research from the station!](#)

To activate the dataset and begin your exploratory audit, install the package repository and initialise the object inside your local workspace session memory:

```
# Install the data package (execute this command once in your console pane and then delete!)
# install.packages("palmerpenguins")

# Load the package data into active memory
library(palmerpenguins)
data("penguins")

# Examine the structure of the dataset - always do this when loading a new dataset!
glimpse(penguins) # tidyverse version (from dplyr package)
str(penguins) # base R version
```

Look closely at your console printout. The `glimpse()` command maps out the entire architectural anatomy of your dataset. Note the variable vector types listed within the arrow markers:

- `<fct>` (Factor): Categorical groupings with fixed levels (e.g., species containing Adelie, Chinstrap, and Gentoo).
- `<dbl>` (Double): Continuous numeric measurements containing decimals (e.g., `bill_length_mm`).
- `<int>` (Integer): Whole number variables tracking counts (e.g., `body_mass_g`).

Before moving into data modification, we can also conduct a statistical overview to map out missing observations and parameter boundaries:

```
# Generate an exploratory summary matrix
summary(penguins)
```

The `summary()` printout is an immediate diagnostic tool. It reports the presence of explicit missing indicators (NA) within individual biological metrics. For instance, if `sex` or `body_mass_g` lists several NA counts, it flags that certain individual penguins escaped the researchers during that day's field sampling!

1.7 Foundational grammar: Slicing, filtering, sorting, and transforming

The `dplyr` package provides a highly structured grammar of data manipulation built around intuitive action verbs. Every function shares a standard design: it accepts an input data frame as its primary argument and outputs a cleanly modified tibble. This section introduces the foundational verbs required to shape tables manually.

1.7.1 Isolating attributes with `select()`

Ecological data tables frequently collect duplicate logistics records that do not contribute to morphological or physiological questions. The `select()` verb allows you to slice your datasets vertically, isolating or dropping columns based on variable names:

```
# Vertically slice specific morphometric variables by explicit name
morphology_metrics <- select(penguins, species, bill_length_mm, bill_depth_mm,
body_mass_g)
glimpse(morphology_metrics)
```

```
# Retain a continuous block of attributes using the colon operator
spatial_block <- select(penguins, species:island)

# Discard logistics tracking attributes while preserving everything else using the
minus sign
clean_scientific_fields <- select(penguins, -year)
```

1.7.2 Sifting rows with filter()

While `select()` targets variables vertically, `filter()` isolates records horizontally based on targeted conditional parameters. R evaluates every row against your logical expressions, retaining entries that evaluate to TRUE and dropping those that return FALSE or NA.

```
# Isolate observations belonging to a single categorical target group
adelie_cohort <- filter(penguins, species == "Adelie")

# Sift out individuals using continuous numerical boundary thresholds
# Preserves only large penguins whose mass exceeds 4500 grams
heavy_penguins <- filter(penguins, body_mass_g > 4500)

# Combine multiple conditional parameters across separate attributes
# Preserves records matching Gentoo penguins sampled explicitly on Biscoe Island
biscoe_gentoo <- filter(penguins, species == "Gentoo" & island == "Biscoe")

# Sift records matching multiple targeting flags within an explicit set
sub_islands <- filter(penguins, island %in% c("Dream", "Torgersen"))
```

1.7.3 Ordering sequences with arrange()

The `arrange()` verb alters the sorting configuration of rows within your data frame without changing individual cell values. This is essential for inspecting size hierarchies or chronologically ordering long-term environmental observation lines:

```
# Sort penguins by ascending body mass (Default setting: Smallest mass first)
lightest_first <- arrange(penguins, body_mass_g)

# Sort penguins in descending sequence using the desc() layout wrapper
```

```
heaviest_first <- arrange(penguins, desc(body_mass_g))

# Execute nested sorting criteria: Group by species, then sort by descending bill
length
stratified_morphology <- arrange(penguins, species, desc(bill_length_mm))
```

1.7.4 Introducing the Pipe (|>)

Up until now, you have been performing data operations one step at a time, often creating "intermediate" variables to hold the result of each stage. While this is great for clarity, it can quickly clutter your environment with objects like `data_v1`, `data_v2`, and `data_v3`.

In R, there is a much cleaner way to chain these operations together: **the pipe**.

Think of the pipe as a "recipe" for your data. Instead of performing an action, saving it, and then taking the next step, you can pass your data through a series of transformations in a single, linear flow.

The Logic of the Pipe:

- **Without a Pipe:** You have to read the code "inside-out." You start in the middle, perform the `mutate()`, then wrap that in a `filter()`, and finally wrap that all in a `select()`.
- **With a Pipe:** You read from left to right, or top to bottom. You take your data, *then* you filter it, *then* you mutate it to create a new column, *then* you select the columns you need.

The syntax for the native R pipe is `|>`. (Note there is also a pipe operator built into the tidyverse package `magrittr`, which is very common `%>%`)

Example: Instead of:

```
penguins_subset <- mutate(penguins, bill_ratio = bill_length_mm / bill_depth_mm)
penguins_final <- filter(penguins_subset, species == "Adelie")
```

You can simply write:

```
penguins_final <- penguins |>
  mutate(bill_ratio = bill_length_mm / bill_depth_mm) |>
  filter(species == "Adelie")
```

By using the pipe, your code becomes easier to read, easier to debug, and much more efficient

to write! For more information, check out this [short video](#).

1.7.5 Computing new attributes with mutate()

Frequently in ecology and marine science in general, we need to calculate new variables based on ones that exist in our raw data. For example this might be to scale observations, apply standard geometric conversions, or compute morphological ratios. In data wrangling we call this “mutating” a new variable and we use the mutate() verb to modify existing attributes or append entirely new vectors to the data frame.

```
# Calculate a new morphological ratio in our environment
penguin_ratios <- penguins |>
  mutate(body_mass_kg = body_mass_g / 1000,    # Convert grams to kilograms
         bill_ratio = bill_length_mm / bill_depth_mm # Bill ratio
  )

# View your newly engineered variables appended to the far-right columns
glimpse(penguin_ratios)
```

1.8 Data aggregation and ecological summarisation

Isolating and sorting individual rows or columns allows you to clean and audit a dataset, but it does not tell you the broader or underlying ecological story. To extract meaningful biological conclusions—such as determining whether morphological allometry varies by species or island group—you must compress thousands of individual observations into explicit population summary metrics.

This is achieved using the dual-layered combination of group_by() and summarise().

When you apply group_by(), R does not alter the physical appearance of the table. Instead, it alters how it interacts with subsequent functions by creating hidden, virtual data buckets based on categorical factor levels. When you immediately follow this with summarise(), R calculates targeted statistical reductions for each individual bucket and collapses them into a brand-new summary tibble.

```
# Grouping our active memory penguins by species
```

```
grouped_penguins <- group_by(penguins, species)

# Notice that the table looks identical, but metadata notes 'Groups: species [3]'
print(grouped_penguins)

# Collapsing the buckets into explicit summary metrics
species_mass_summary <- summarise(grouped_penguins,
  mean_mass_g = mean(body_mass_g)
)

print(species_mass_summary)
```

Look closely at your console printout. The summary operation failed and returned an entire column of NA values for certain groups! Why is that!?

This is an essential behavioral design feature of R known as the **Missing Value Trap**. If a column contains even a single missing observation flag (NA), any mathematical aggregation function (such as mean(), sd(), or sum()) will automatically return NA to protect you from miscalculating metrics on incomplete data. To override this safely, you must explicitly declare that missing cells should be dropped during calculation. Here we'll now use the magrittr pipe to wrangle our data, avoiding the missing value trap.

```
# Overcoming the missing value trap using na.rm = TRUE
biological_signal <- penguins %>%
  group_by(species, sex) %>%
  summarise(
    sample_size = n(), # Count total
    individuals per category
    mean_mass_g = mean(body_mass_g, na.rm = TRUE), # Calculate mean
    ignoring missing cells
    sd_mass_g = sd(body_mass_g, na.rm = TRUE) # Standard deviation
    calculation
  )

print(biological_signal)
```

Have a look at the resulting table structure. By combining these verbs, you have successfully compressed a long observational log into a tight, clean overview of sexual dimorphism and species variation across the study area.

1.9 Integrating data grammar with visual diagnostics in qmd

Data manipulation and data visualisation are not separate tasks; they are two sides of the same

analytical coin. In professional marine science, you wrangle data specifically to prepare it for visual display, and you use plots to audit your data engineering logic.

As we learned in *Programming fundamentals*, the tidyverse handles data visualisation using the ggplot2 library, which is built on a conceptual framework called the **Grammar of Graphics**. This framework dictates that every chart is constructed layer-by-layer:

1. **The Data Layer:** Declaring the source dataset table object.
2. **The Aesthetic Mapping (aes):** Defining which variables are mapped to structural axes, colors, shapes, or sizes.
3. **The Geometric Layer (geom_...):** Defining the visual shape that represents the numbers (e.g., points, bars, lines, or boxplots).

You should now have a code base in your R script that wrangles the penguin data into a clean state. From here we can shift from basic data exploration into polished communication. In *Programming Fundamentals*, you learned that a script (.R) is perfect for the "messy" stages of data cleaning. However, for a professional ePortfolio, we need a format that seamlessly combines our code, our logic, and our final outputs.

This is where **Quarto (.qmd)** becomes a very valuable tool.

By using your provided analysis.qmd template, you can "source" the data cleaning work you have already completed and render your results directly into a professional HTML document.

1.9.1 Wrangling and Plotting in Parallel

You have already mastered the art of the pipe (`|>`) and the basics of ggplot2. Now, we will combine these into a single "reproducible block."

Because your .qmd already loads your cleaned data via `source("Workshop1.R")`, you can build your visualisations and summary tables in real-time. The goal here is to present your results side-by-side.

Challenge: Visualising Body Size and Shape

Using your loaded data, create a code chunk in your .qmd that performs the following two tasks side-by-side:

1. **Summary Table:** Pipe your data to `group_by()` and `summarise()` to calculate the mean body mass for each species and island.
2. **Visualisation:** Pipe the same dataset into `ggplot()` to create a boxplot of `body_mass_g` by species, with the island variable mapped to fill or `facet_wrap()`.

1.9.2 Piping Directly to Visualisation

Sometimes, you do not need to save a summary table as an object. If your goal is to visualise patterns and uncertainty within your aggregated data, you can extend your pipeline directly into ggplot2. This is a powerful, efficient workflow for rapid exploratory data analysis (EDA).

Example: Visualising Mean Trends with Uncertainty Instead of creating a summary table object first, you can pipe your grouped data straight into a plot and use `geom_errorbar()` to represent the spread of your data:

```
# Pipe directly from aggregation to plotting with error bars
mass_compare_plot <- penguins |>
  group_by(species, island) |>
  summarise(
    mean_mass = mean(body_mass_g, na.rm = TRUE),
    sd_mass = sd(body_mass_g, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) |>
  ggplot(aes(x = species, y = mean_mass, colour = island)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = mean_mass - sd_mass,
                    ymax = mean_mass + sd_mass),
                width = 0.2) +
  labs(title = "Mean Body Mass by Species and Island",
        subtitle = "Error bars represent standard deviation",
        y = "Mean Body Mass (g)",
        x = "Species") +
  theme_minimal()

mass_compare_plot
```

Pause for thought: Think about what this figure is telling you about the biology of these species of penguins on these different islands. What do you think a statistical test would tell you here?

Challenge:

In your .qmd template, try creating a new plot. Experiment with swapping `sd_mass` (standard deviation) for standard error—you will need to divide your `sd` by the square root of `n` (`sd / sqrt(n)`). See how that changes your visual interpretation of the data uncertainty. Present the summary table alongside the associated figure and write down your thoughts on the underlying ecological story that the data are telling.

Decision Guide: Parallel vs. Direct Piping

When choosing your workflow, use this guide to ensure your project remains organised:

Approach	Best Used When...
Parallel (1.9.1)	You need to save the summary table (as a .csv or .rds file) for inclusion in your final report or to share with team members.
Direct Piping (1.9.2)	You are in the exploration phase , iterating through visualisations rapidly to find the most effective way to communicate your findings.

1.10 Saving, exporting, and version milestones

Excellent work team! You have now successfully ingested datasets, performed exploratory audits, engineered summary metrics, and generated figures. As long as you have saved all your code, you can safely close down your R project and your scripts and qmd files will be waiting for you next time you launch the project (think about committing and pushing to GitHub though!)

In future, you may encounter wrangling and analysis that takes significant processing power and memory to process, in which case you may want to save your outputs. Before we do that we need to check if we have the correct folders, and if not we can create them:

```
# Create output directories if they do not exist:
if (!dir.exists("outputs/figures")) dir.create("outputs/figures") # folder for figs
if (!dir.exists("outputs/tables")) dir.create("outputs/tables") # folder for tables
if (!dir.exists("Rdata")) dir.create("Rdata") # folder for Rdata objects
```

In this workshop we have created both summary tables and figures and we have a few options for saving these objects.

Tables

- **As a Universal CSV text file:** Ideal for sharing data tables with collaborators who do not

use R.

- **As a Native R Binary RDS object:** Ideal for storing clean, complex intermediate tables for your own subsequent workshops. Saving as an RDS file preserves your exact vector configurations (such as custom categorical factor levels and numeric type definitions), meaning you never have to re-format column types when you reload the data next week.

```
# 1. Exporting our collapsed summary table as a universal flat text file
write_csv(biological_signal, "outputs/penguin_species_mass_summary.csv")

# 2. Saving our cleaned morphological cohort table as a native R binary file
saveRDS(clean_cohort, "outputs/clean_penguin_morphology_cohort.rds")
```

Figures

ggplot2 has an in-built function for saving figures - `ggsave()`:

```
ggsave("outputs/mass_compare_plot.png",
       plot = mass_compare_plot,
       width = 120, height = 120,
       units = "mm", dpi = 300)
```

Spend the final twenty minutes of today's lab session making sure your work is saved and populated to your GitHub repository and ePortfolio:

Step 1: Upstream Git Synchronisation

Use the interactive RStudio Git pane in the upper-right corner to track your files. You can use a commit message along the lines of:

“Complete Workshop 1 manual tidyverse grammar and penguin data audit”

Step 2: Digital ePortfolio Compilation

Launch your custom Google Sites management console and navigate to your active professional profile space. Create a dedicated child landing page under your main page hierarchy titled **Module: R for Marine Science**. Within this module block, build your specific sub-page entry: **Workshop 1: Foundations of Marine Data Science**.

Your published site page should contain the following structural reflections:

- A brief, 3-sentence summary of the dataset you have worked on and wrangled during

today's session.

- An embedded html file which includes your finished morphological plot figure, complete with clear axis titles, measurement units, and legends, along with relevant accompanying tables.
- A reflective analytical paragraph detailing your personal experience during this **AI-Off** phase. Detail how writing code manually forced you to inspect console error messages, track variable classes, and identify the specific differences between a legacy data frame and a modern tidyverse tibble.